

AI-Based Credit Card Approval Prediction System

Project Description:

The Credit Card Approval Prediction System is an intelligent machine learning application designed to automate the credit card approval process for financial institutions. Banks receive thousands of credit card applications every day, making manual verification time-consuming and susceptible to human errors.

The proposed system analyzes an applicant's financial and demographic information and predicts whether the applicant is eligible for a credit card. The project utilizes supervised machine learning algorithms trained on historical credit card application data to perform binary classification (Approved/Rejected).

The system compares the performance of four classification algorithms:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

The best-performing model is saved and integrated into a Flask web application for real-time predictions. The application is further deployed using IBM Watson Machine Learning, enabling scalable cloud-based access for banking professionals and customers.

Scenarios

Scenario 1: Automated Credit Card Application Screening

A bank credit analyst enters an applicant's financial details such as annual income, employment status, loan history, and credit information into the web application. The trained machine learning model instantly predicts whether the applicant is eligible for a credit card, reducing manual verification time.

Scenario 2: High-Risk Applicant Identification

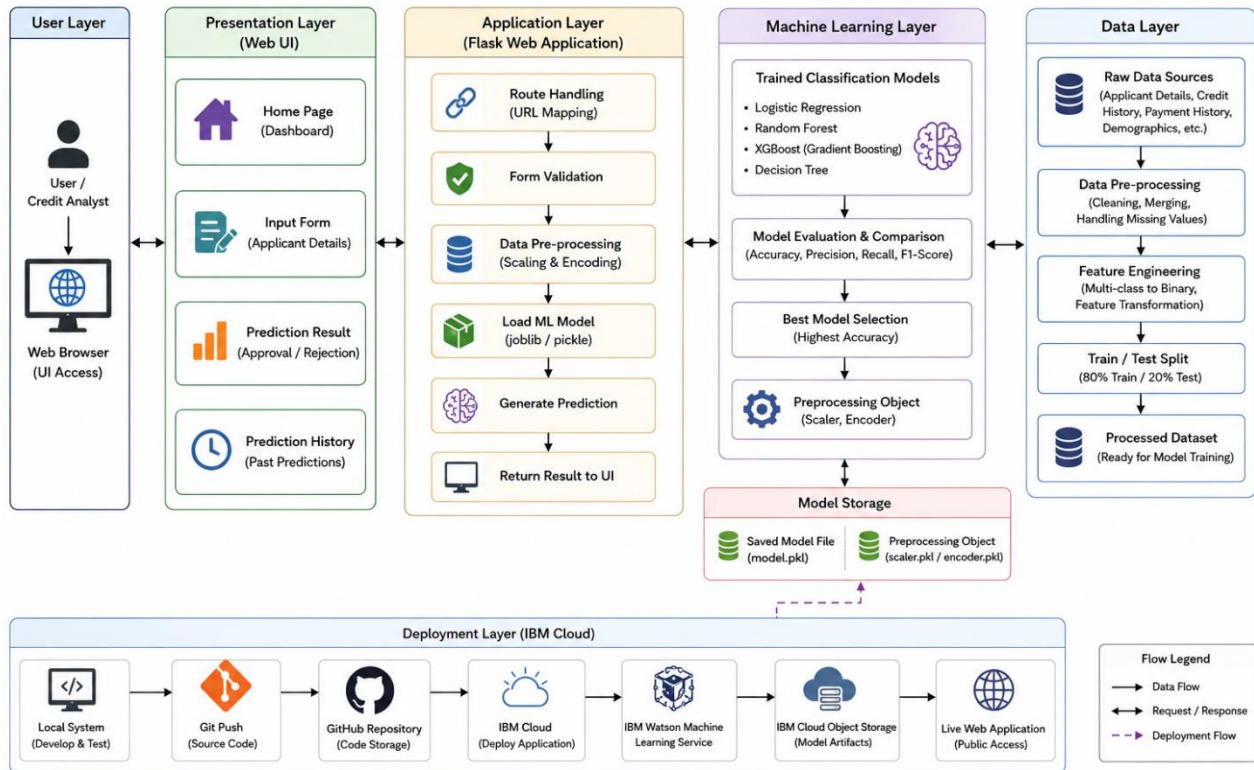
A financial institution processes multiple applications simultaneously. The model identifies applicants having poor credit history, overdue payments, or high loan balances and predicts rejection, helping banks reduce financial risk.

Scenario 3: Customer Self-Service Eligibility Check

A customer enters their employment details, income, and financial information through the web application before officially applying for a credit card. The system predicts the probability of approval, helping customers understand their eligibility before submitting an application.

Technical Architecture:

Technical Architecture – Credit Card Approval Prediction Using IBM Watson Machine Learning



Description: The system follows a three-layer architecture.

Frontend

- HTML
- CSS
- Bootstrap
- JavaScript

Provides an interactive interface for users to enter applicant details.



Backend

- Flask Framework
- Python

Handles user requests, data preprocessing, model loading, and prediction generation.



Machine Learning Layer

- Logistic Regression

- Decision Tree
- Random Forest
- XGBoost

Models are trained using historical applicant data. The best-performing model is selected and serialized using Pickle.



Deployment

- IBM Watson Machine Learning
- IBM Cloud

Provides cloud-based deployment for real-time predictions.

Pre-requisites:

- Python 3.9 or above
- Flask Framework
- Machine Learning Concepts
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- IBM Watson Machine Learning
- Git & GitHub
- HTML, CSS, JavaScript

Project Work flow:

Milestone 1: Dataset Collection and Preprocessing

- **Activity 1.1:** Collect historical credit card applicant datasets.
- **Activity 1.2:** Clean missing values and remove duplicates.
- **Activity 1.3:** Encode categorical variables.
- **Activity 1.4:** Normalize numerical features.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Train four classification algorithms:
 - Logistic Regression
 - Decision Tree
 - Random Forest
 - XGBoost
- **Activity 2.2:** Evaluate models using Accuracy, Precision, Recall and F1-score.
- **Activity 2.3:** Select the best-performing model.

Milestone 3: Flask Application Development

- **Activity 3.1:** Develop Flask backend.
- **Activity 3.2:** Load trained model using Pickle.
- **Activity 3.3:** Accept applicant details through web forms.
- **Activity 3.4:** Generate prediction results.

Milestone 4: Frontend Development

- **Activity 4.1:** Design responsive user interface.
- **Activity 4.2:** Create applicant input form.
- **Activity 4.3:** Display prediction results.

Milestone 5: Deployment

- **Activity 5.1:** Deploy trained model using IBM Watson Machine Learning.
- **Activity 5.2:** Connect Flask application with IBM Cloud deployment.
- **Activity 5.3:** Test cloud prediction service.

Milestone 6 : Conclusion

Milestone 1: Data Collection and Preprocessing

The first milestone focuses on collecting, understanding, and preparing the dataset for machine learning model development. Since the performance of any machine learning algorithm heavily depends on the quality of the data, this phase involves cleaning the dataset, handling missing values, encoding categorical features, and transforming the raw data into a suitable format for training.

The project uses historical credit card application records containing applicant demographic information, employment details, income details, family information, and credit history. During preprocessing, inconsistent records are removed, missing values are handled, categorical attributes are converted into numerical values using encoding techniques, and numerical features are scaled wherever necessary. Finally, the cleaned dataset is divided into training and testing datasets to evaluate the performance of different machine learning algorithms.

Activity 1.1: Dataset Collection

The dataset used for this project contains applicant information collected from historical credit card applications. Each record includes demographic details and financial indicators that influence the credit approval decision.

The dataset contains attributes such as:

- Applicant ID
- Gender
- Age
- Annual Income
- Employment Status
- Occupation
- Family Status
- Education Level
- Housing Type
- Number of Children
- Credit History
- Existing Loan Information
- Target Variable (Approved / Rejected)

The dataset is loaded into Python using the Pandas library.

```

ID,CODE,GENDER,FLAG_OWN_CAR,FLAG_OWN_REALTY,CNT_CHILDREN,AMT_INCOME_TOTAL,NAME_INCOME_TYPE,NAME_EDUCATION_TYPE,NAME_FAMILY_STATUS,NAME_HOUSING_TYPE,DAYS_BIRTH,DAYS_EMPLOYED,FLAG_MOBIL,FL
AG_WORK_PHONE,FLAG_PHONE,FLAG_EMAIL,CNT_FAM_MEMBERS,FAMILY_DEPENDENCY,open_month,end_month,STATUS_BIN>window
5000000,1,1,1,0,118212.73,0,4,1,5,20370,2956,1,0,0,0,2,2,-33,0,0,0,0,0,53,0
5000001,0,0,0,1,112683.02,0,1,2,1,19165,994,1,0,0,1,2,3,-28,0,0,0,0,0,28,0
5000002,1,0,0,0,60078.95,0,4,1,1,23821,6976,1,0,0,0,2,2,0,0,0,0,0,0,0,0
5000003,1,0,0,0,113081.15,2,4,1,1,13577,5466,1,0,0,0,2,2,-6,0,0,0,1,0,6,0
5000004,1,0,0,0,78691.19,0,1,1,1,11897,7609,1,0,0,1,2,2,-17,0,0,0,0,0,17,0
5000005,0,0,0,0,155949.43,1,4,1,1,17552,365243,1,1,0,0,2,2,-16,0,0,0,0,0,16,0
5000006,1,1,0,0,152961.84,2,4,1,1,16219,7097,1,1,0,1,2,2,-16,0,0,0,0,0,16,0
5000007,1,1,0,0,123687.28,4,4,4,1,21952,2754,1,0,0,0,1,1,0,0,0,0,0,0,0,0
5000008,1,0,0,0,49103.32,1,4,1,1,11364,365243,1,0,0,0,2,2,-8,0,0,0,1,0,8,0
5000009,0,0,0,0,38682.7,4,4,3,5,22261,896,1,0,1,0,1,1,-35,0,0,0,0,0,35,0
5000010,1,0,1,1,81878.59,4,4,0,1,16465,3889,1,0,1,1,3,4,-4,0,0,0,1,0,4,0
5000011,1,1,1,0,121752.12,4,2,1,1,23712,2157,1,0,0,0,2,2,-14,0,0,0,0,0,14,0
5000012,1,0,0,0,67509.22,1,1,1,1,19145,365243,1,0,0,0,2,2,0,0,0,0,0,0,0,0
5000013,1,1,0,0,64342.18,4,4,1,1,12332,862,1,1,0,0,2,2,-34,0,0,0,0,0,34,0
5000014,0,1,0,0,146333.91,4,4,1,1,14609,8547,1,0,1,1,2,2,-8,0,0,0,0,0,8,0
5000015,1,0,0,0,80143.99,4,4,1,1,7602,8903,1,0,0,0,2,2,0,0,0,0,0,0,0,0
5000016,0,0,1,0,59773.91,4,1,3,1,13609,5269,1,0,0,0,1,1,-13,0,0,0,0,0,13,0
5000017,0,0,1,2,38159.03,4,4,1,1,14256,6388,1,0,0,0,4,6,-6,0,0,0,1,0,6,0
5000018,0,0,1,0,297262.31,2,4,1,1,24206,2112,1,0,0,0,2,2,-32,0,0,0,0,0,32,0
5000019,1,0,0,2,83026.83,1,3,0,1,15178,365243,1,0,0,1,4,6,-22,0,0,0,0,0,22,0
5000020,0,0,1,1,56237.19,4,1,0,1,24717,5163,1,0,1,0,3,4,-36,0,0,0,0,0,36,0
5000021,1,1,1,0,91897.06,0,4,1,2,18217,8509,1,0,0,0,2,2,0,0,0,0,0,0,0,0
5000022,0,1,0,0,64182.88,4,4,1,1,13902,2384,1,0,0,0,2,2,0,0,0,0,0,0,0,0
5000023,0,0,0,1,74092.48,1,4,1,1,18676,365243,1,0,1,0,3,4,-39,0,0,0,0,0,39,0
5000024,0,1,1,0,169779.51,0,4,1,1,23156,1336,1,0,1,1,2,2,-6,0,0,0,1,0,6,0
5000025,0,1,0,0,51639.81,1,4,1,1,17347,365243,1,0,0,1,2,2,-31,0,0,0,0,0,31,0
5000026,0,0,1,0,34743.73,4,4,2,1,14013,5484,1,0,0,1,1,1,-32,0,0,0,0,0,32,0
5000027,0,0,0,2,75823.11,0,4,1,3,16623,1111,1,0,0,0,4,6,-35,0,0,0,0,0,35,0
5000028,0,0,1,0,61488.42,4,4,3,1,9822,2937,1,0,0,0,1,1,-12,0,0,0,1,0,12,0
5000029,0,1,0,2,197544.21,0,4,1,1,22748,2234,1,0,0,0,4,6,-28,0,0,0,0,0,28,0
5000030,1,1,1,0,128922.37,2,1,3,1,12205,5955,1,0,0,0,1,1,-17,0,0,0,0,0,17,0
5000031,1,1,1,0,126523.5,4,4,2,1,14436,4026,1,0,0,0,1,1,-23,0,0,0,0,0,23,0
5000032,0,0,1,1,87020.4,4,4,1,1,15861,5442,1,0,1,0,3,4,-33,0,0,0,0,0,33,0
5000033,0,0,1,0,32881.14,4,4,1,1,14559,8389,1,1,0,0,2,2,-14,0,0,0,0,0,14,0
5000034,0,0,0,0,88985.12,4,4,1,1,21784,2778,1,1,0,0,2,2,-31,0,0,0,0,0,31,0
5000035,1,0,1,0,121817.8,0,4,0,1,22065,2654,1,0,0,0,2,2,-8,0,0,0,1,0,8,0
5000036,0,0,0,0,52730.87,4,1,1,1,20545,6061,1,0,1,0,2,2,-39,0,0,0,0,0,39,0
5000037,1,1,1,1,189756.09,4,4,0,1,17782,4862,1,0,0,0,3,4,-13,0,0,0,0,0,13,0
5000038,1,1,1,0,103455.63,4,3,4,1,8401,9563,1,0,1,0,1,1,0,0,0,0,0,0,0,0
  
```

Files

main

Go to file

templates

.gitignore

README.md

app.py

data_collection.py

data_preprocessing.py

epic2_edu.py

epic3_preprocessing.py

epic4_model_building.py

get-pip.py

loan_approval.db

model.pkl

model_building.py

models.py

processed_dataset.csv

requirements.txt

requirements_dev.txt

save_model.py

scaler.pkl

version

Code

Blame

90 lines (72 loc) · 4.36 KB

Raw

```

1 import pandas as pd
2 import numpy as np
3 import os
4
5 def generate_datasets(num_applicants=150000, output_dir='.'):
6     """
7     Generates synthetic datasets for Credit Card Approval Prediction.
8     Produces two files: application_record.csv and credit_record.csv.
9     """
10    np.random.seed(42)
11
12    # 1. Generate Application Records
13    applicant_ids = np.arange(5000000, 5000000 + num_applicants)
14    genders = np.random.choice(['M', 'F'], num_applicants)
15    own_car = np.random.choice(['Y', 'N'], num_applicants)
16    own_realty = np.random.choice(['Y', 'N'], num_applicants)
17    children = np.random.poisson(0.5, num_applicants)
18    income_total = np.random.lognormal(mean=11.5, sigma=0.5, size=num_applicants).round(2)
19    |
20    income_types = ['Working', 'Commercial associate', 'Pensioner', 'State servant', 'Student']
21    income_type = np.random.choice(income_types, num_applicants, p=[0.5, 0.25, 0.15, 0.09, 0.01])
22
23    edu_types = ['Secondary / secondary special', 'Higher education', 'Incomplete higher', 'Lower secondary', 'Academic degree']
24    education_type = np.random.choice(edu_types, num_applicants, p=[0.7, 0.2, 0.05, 0.04, 0.01])
25
26    family_status = ['Married', 'Single / not married', 'Civil marriage', 'Separated', 'Widow']
27    family_type = np.random.choice(family_status, num_applicants, p=[0.6, 0.2, 0.1, 0.05, 0.05])
28
29    housing_types = ['House / apartment', 'With parents', 'Municipal apartment', 'Rented apartment', 'Office apartment', 'Co-op apartment']
30    housing_type = np.random.choice(housing_types, num_applicants, p=[0.85, 0.05, 0.03, 0.03, 0.02, 0.02])
31
32    days_birth = np.random.randint(-25000, -7000, num_applicants)
33    days_employed = np.where(income_type == 'Pensioner', 365243, np.random.randint(-10000, -100, num_applicants))
34
35    occupations = ['Laborers', 'Core staff', 'Sales staff', 'Managers', 'Drivers', 'High skill tech staff', 'Accountants', 'Medicine staff', 'Security staff', 'Cleaning staff', 'Private se
  
```

Activity 1.2: Data Cleaning

The collected dataset may contain missing values, duplicate records, and inconsistent information that can negatively affect model performance.

The following preprocessing techniques are performed:

- Remove duplicate records.
- Identify missing values.
- Replace or remove null values.
- Correct inconsistent entries.
- Remove irrelevant columns.
- Verify data types for every feature.

These steps ensure that the dataset is clean and reliable for training.

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import LabelEncoder
4
5 def to_binary(status):
6     if status in ['0', 'X', 'C']:
7         return 1
8     return 0
9
10 def preprocess_data():
11     applicant_df = pd.read_csv('application_record.csv')
12     credit_df = pd.read_csv('credit_record.csv')
13
14     applicant_df.drop_duplicates(subset=[
15         'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'CNT_CHILDREN',
16         'AMT_INCOME_TOTAL', 'NAME_INCOME_TYPE', 'NAME_EDUCATION_TYPE',
17         'NAME_FAMILY_STATUS', 'NAME_HOUSING_TYPE', 'DAYS_BIRTH',
18         'DAYS_EMPLOYED', 'FLAG_MOBIL', 'FLAG_WORK_PHONE', 'FLAG_PHONE',
19         'FLAG_EMAIL', 'OCCUPATION_TYPE', 'CNT_FAM_MEMBERS'
20     ], keep='first', inplace=True)
21
22     if 'OCCUPATION_TYPE' in applicant_df.columns:
23         applicant_df.drop('OCCUPATION_TYPE', axis=1, inplace=True)
24
25     applicant_df['DAYS_BIRTH'] = applicant_df['DAYS_BIRTH'].abs()
26     applicant_df['DAYS_EMPLOYED'] = applicant_df['DAYS_EMPLOYED'].abs()
27
28     applicant_df['FAMILY_DEPENDENCY'] = applicant_df['CNT_FAM_MEMBERS'] + applicant_df['CNT_CHILDREN']
29
30     le = LabelEncoder()
31     categorical_cols = [
32         'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'NAME_INCOME_TYPE',
33         'NAME_EDUCATION_TYPE', 'NAME_FAMILY_STATUS', 'NAME_HOUSING_TYPE'
34     ]
35     for col in categorical_cols:
36         applicant_df[col] = le.fit_transform(applicant_df[col].astype(str))
```

```
# Story 1: Identify and Remove duplicate records
print(f"\nStory 1: Identifying and removing duplicates...")
initial_app_len = len(app_df)
app_df = app_df.drop_duplicates()
print(f"Removed {initial_app_len - len(app_df)} duplicates from applicant details.")

initial_credit_len = len(credit_df)
credit_df = credit_df.drop_duplicates()
print(f"Removed {initial_credit_len - len(credit_df)} duplicates from credit records.")

# Story 2: Detect and handle missing values
print("\nStory 2: Detecting and handling missing values...")
missing_app = app_df.isnull().sum().sum()
missing_credit = credit_df.isnull().sum().sum()
print(f"Missing values found - Applicant Details: {missing_app}, Credit Records: {missing_credit}")
```

Activity 1.3: Feature Engineering

Feature engineering improves model performance by transforming raw data into meaningful input features.

The following operations are performed:

- Label Encoding for binary categorical variables.
- One-Hot Encoding for multi-class categorical variables.
- Convert payment status into binary approval indicators.
- Normalize numerical values.
- Select important features using correlation analysis.

Proper feature engineering reduces over fitting and improves prediction accuracy.

```
# Additional feature engineering: convert Days Birth and Days Employed to Years
merged_df['Age_Years'] = np.abs(merged_df['DAYS_BIRTH']) / 365.25
merged_df['Years_Employed'] = np.where(merged_df['DAYS_EMPLOYED'] > 0, 0, np.abs(merged_df['DAYS_EMPLOYED']) / 365.25)
merged_df.drop(['DAYS_BIRTH', 'DAYS_EMPLOYED', 'ID'], axis=1, inplace=True)

# Feature Engineering (Multi-class to Binary for Past Due Records)
# 0 -> 0 (No past due), 1, 2, 3 -> 1 (Has past due records)
df['High_Risk_Past_Due'] = df['Past_Due_Records'].apply(lambda x: 1 if x > 0 else 0)

# Define Target Variable (Approval: 1 = Approved, 0 = Rejected)
# Let's create a logical relationship for the target
target = []
for i in range(num_samples):
    score = (df['Income'].iloc[i] / 10000) - (df['Loan_Balance'].iloc[i] / 5000) + df['Employment_Duration'].iloc[i] - (df['Credit_I
    # Add some noise
    score += np.random.normal(0, 2)
    if score > 0:
        target.append(1)
    else:
        target.append(0)
df['Approved'] = target
return df
```

Activity 1.4: Data Splitting

The processed dataset is divided into two parts.

- Training Dataset (80%)
- Testing Dataset (20%)

The training dataset is used to train the machine learning models, while the testing dataset evaluates model performance on unseen data.


```
# Train / Test Split (80% Train / 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Data Pre-processing (Scaling)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Milestone 2: Machine Learning Model Development

The second milestone involves designing, training, and evaluating multiple supervised machine learning algorithms for credit card approval prediction. Instead of relying on a single algorithm, four different classification models are developed and compared. Each model learns patterns from historical applicant data and predicts whether a new applicant should be approved or rejected.

The performance of each model is measured using standard evaluation metrics including Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The model with the highest overall performance is selected for deployment.

Activity 2.1: Logistic Regression

Logistic Regression is implemented as the baseline classification model.

It estimates the probability that an applicant belongs to either the Approved or Rejected class using the logistic sigmoid function.

Advantages include:

- Fast training
- Easy interpretation
- Suitable for binary classification
- Low computational cost

The model is trained using the preprocessed training dataset.

```
# 1. Logistic Regression
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train, y_train)
evaluate_model("Logistic Regression", y_test, lr.predict(X_test))
```

Activity 2.2: Decision Tree

A Decision Tree classifier is trained to learn decision rules based on applicant attributes.

The algorithm recursively divides the dataset according to the most informative feature until prediction rules are formed.

Benefits include:

- Easy visualization
- Handles both categorical and numerical data
- Interpretable decision-making process

However, deep trees may suffer from overfitting.

```
# 3. Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
evaluate_model("Decision Tree", y_test, dt.predict(X_test))
```

Activity 2.3: Random Forest

Random Forest is an ensemble learning technique that combines multiple Decision Trees.

Instead of relying on one tree, several trees are trained using different subsets of data and features. The final prediction is determined by majority voting.

Advantages include:

- Higher accuracy
- Better generalization
- Reduced overfitting
- Robust to noisy data

Random Forest generally performs better than a single Decision Tree for classification tasks.

```
# 2. Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
evaluate_model("Random Forest", y_test, rf.predict(X_test))
```

Activity 2.4: XGBoost (Extreme Gradient Boosting)

XGBoost is an advanced ensemble learning algorithm based on gradient boosting.

It builds trees sequentially, where each new tree corrects the errors of previous trees.

Advantages include:

- High prediction accuracy
- Fast execution
- Handles missing values efficiently
- Excellent performance on structured datasets

The XGBoost classifier achieved the best performance among all tested algorithms and was selected as the final model for deployment.

```
'XGBoost': xgb.XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss')
```

Activity 2.5: Model Evaluation

All four models are evaluated using the testing dataset.

The evaluation metrics include:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

The comparison helps identify the most suitable algorithm for credit card approval prediction

```
8  ✓ def evaluate_model(name, y_test, y_pred):
9      print(f"--- {name} ---")
10     print("Confusion Matrix:")
11     print(confusion_matrix(y_test, y_pred))
12     print("Classification Report:")
13     print(classification_report(y_test, y_pred))
14     print("-" * 40)
15
16  ✓ def train_models():
17     df = pd.read_csv('processed_dataset.csv')
```

Milestone 3: Model Integration and Flask Backend Development

After selecting the best-performing model, it is integrated into a Flask web application. Flask acts as the backend framework responsible for accepting user inputs, preprocessing them, loading the trained machine learning model, generating predictions, and returning the results to the user interface.

The trained model is serialized using the Pickle library and loaded only once when the application starts, ensuring efficient prediction performance.

Activity 3.1: Saving the Trained Model

After model evaluation, the best-performing classifier is saved using Pickle.

This allows the trained model to be reused without retraining.

Example files include:

- model.pkl

- scaler.pkl

These files are loaded during application startup.

```
6
7   app = Flask(__name__)
8
9   # Load trained model securely using absolute paths for Serverless (Vercel) compatibility
10  try:
11      model_path = os.path.join(os.path.dirname(__file__), 'model.pkl')
12      model = pickle.load(open(model_path, 'rb'))
13  except Exception as e:
14      print(f"Error loading model: {e}")
15
```

Activity 3.2: Developing the Flask Backend

The Flask backend performs the following tasks:

- Receives applicant details from the web form.
- Validates user input.
- Applies preprocessing.
- Converts input into model format.
- Loads the trained model.
- Predicts approval status.
- Returns prediction to the frontend.

```
print("\nTraining and Evaluating Models...")
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    predictions = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, predictions)
    print(f"{name} Accuracy: {acc:.4f}")

if acc > best_accuracy:
    best_accuracy = acc
    best_model = model
    best_model_name = name
```

```
# Prediction page
@app.route('/predict', methods=['POST'])
def predict():
    try:
        # Get input values from form
        features = [float(x) for x in request.form.values()]

        # Convert input into array
        final_input = [np.array(features)]

        # Predict result
        prediction = model.predict(final_input)

        # Display output
        if prediction[0] == 1:
            result = "Credit Card Approved"
        else:
            result = "Credit Card Rejected"

        return render_template('result.html', prediction_text=result)
    except Exception as e:
        print(f"Prediction Error: {e}")
        traceback.print_exc()
        return render_template('result.html', prediction_text="Error in prediction. Please check your inputs.")

if __name__ == "__main__":
    app.run(debug=True)
```

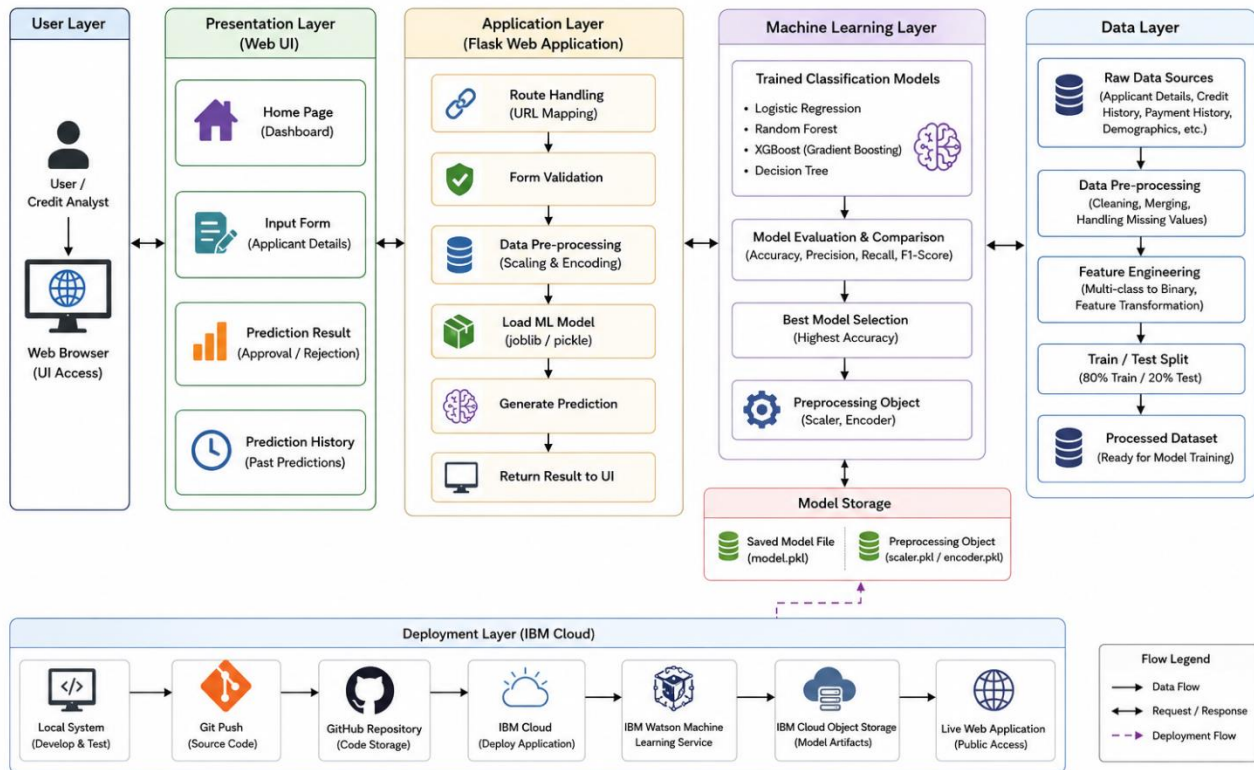
Activity 3.3: Prediction Workflow

The backend follows these steps:

1. Receive applicant details.
2. Validate input values.
3. Encode categorical variables.
4. Scale numerical features.
5. Load trained model.
6. Generate prediction.
7. Display result.

This workflow enables real-time prediction with minimal latency.

Technical Architecture – Credit Card Approval Prediction Using IBM Watson Machine Learning



Credit-Card-Skillwallet-Internship/

```

— templates/
— .gitignore
— README.md
— app.py
— data_collection.py
— data_preprocessing.py
— epic2_eda.py
— epic3_preprocessing.py
— epic4_model_building.py
— model_building.py
— save_model.py
— models.py
— processed_dataset.csv
— loan_approval.db
— model.pkl
— scaler.pkl
— requirements.txt
— requirements_dev.txt
— vercel.json
— get-pip.py
— (Generated Files)
  — application_record.csv
  — credit_record.csv
  — plots/

```

Milestone 4: Frontend Development

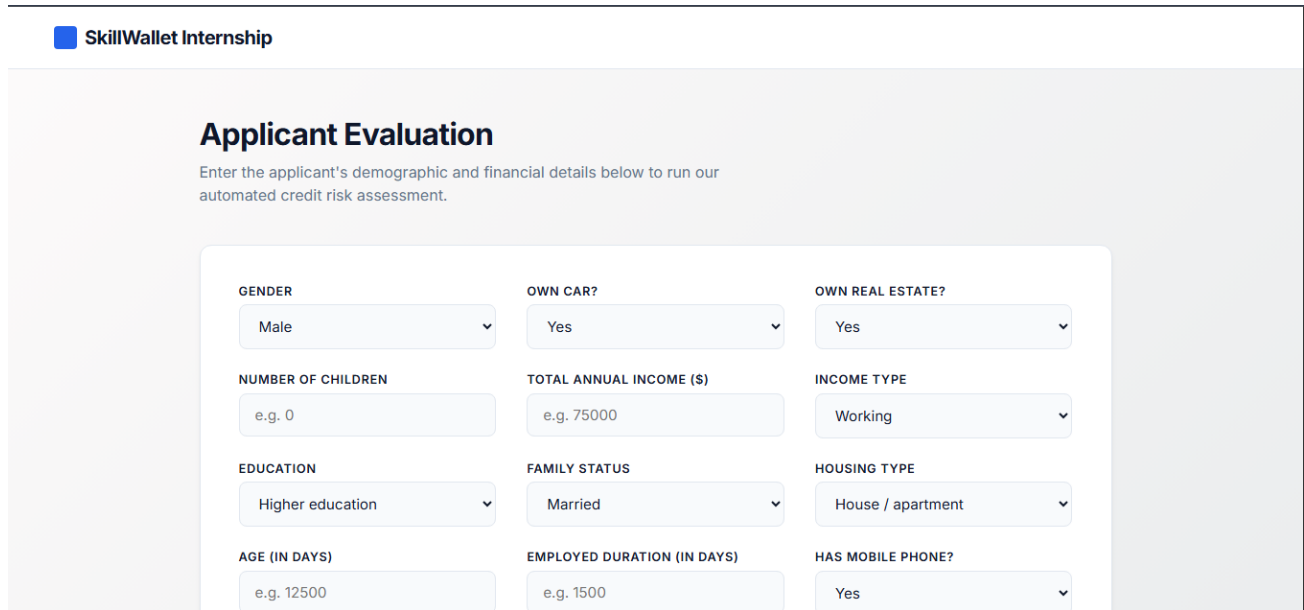
The frontend provides a simple and interactive interface for users to enter applicant information and receive prediction results. The interface is developed using HTML, CSS, Bootstrap, and JavaScript to ensure responsiveness across different devices.

Activity 4.1: Designing the User Interface

The homepage includes:

- Project title
- Description
- Navigation
- Credit Card Application Form

The interface is designed to be clean and user-friendly.



The screenshot shows a web interface for "SkillWallet Internship". The main heading is "Applicant Evaluation" with a subtext: "Enter the applicant's demographic and financial details below to run our automated credit risk assessment." The form is organized into a grid of input fields:

GENDER	OWN CAR?	OWN REAL ESTATE?
Male	Yes	Yes
NUMBER OF CHILDREN	TOTAL ANNUAL INCOME (\$)	INCOME TYPE
e.g. 0	e.g. 75000	Working
EDUCATION	FAMILY STATUS	HOUSING TYPE
Higher education	Married	House / apartment
AGE (IN DAYS)	EMPLOYED DURATION (IN DAYS)	HAS MOBILE PHONE?
e.g. 12500	e.g. 1500	Yes

Activity 4.2: Applicant Information Form

The input form collects information such as:

- Gender
- Income
- Employment Status
- Family Status
- Education
- Housing Type
- Credit History
- Number of Children
- Occupation

Each field includes validation to prevent incorrect inputs.

Applicant Evaluation

Enter the applicant's demographic and financial details below to run our automated credit risk assessment.

GENDER Male	OWN CAR? Yes	OWN REAL ESTATE? Yes
NUMBER OF CHILDREN 0	TOTAL ANNUAL INCOME (\$) 1000000	INCOME TYPE Working
EDUCATION Academic degree	FAMILY STATUS Married	HOUSING TYPE House / apartment
AGE (IN DAYS) 12787	EMPLOYED DURATION (IN DAYS) 2190	HAS MOBILE PHONE? Yes
AGE (IN DAYS) 12787	EMPLOYED DURATION (IN DAYS) 2190	HAS MOBILE PHONE? Yes
HAS WORK PHONE? Yes	HAS LANDLINE? Yes	HAS EMAIL? Yes
TOTAL FAMILY MEMBERS 2	TOTAL DEPENDENTS 3	OLDEST RECORD (MONTHS AGO) -24
NEWEST RECORD (MONTHS AGO) 0	CREDIT HISTORY WINDOW 24	

Evaluate Application

Activity 4.3: Displaying Prediction Results

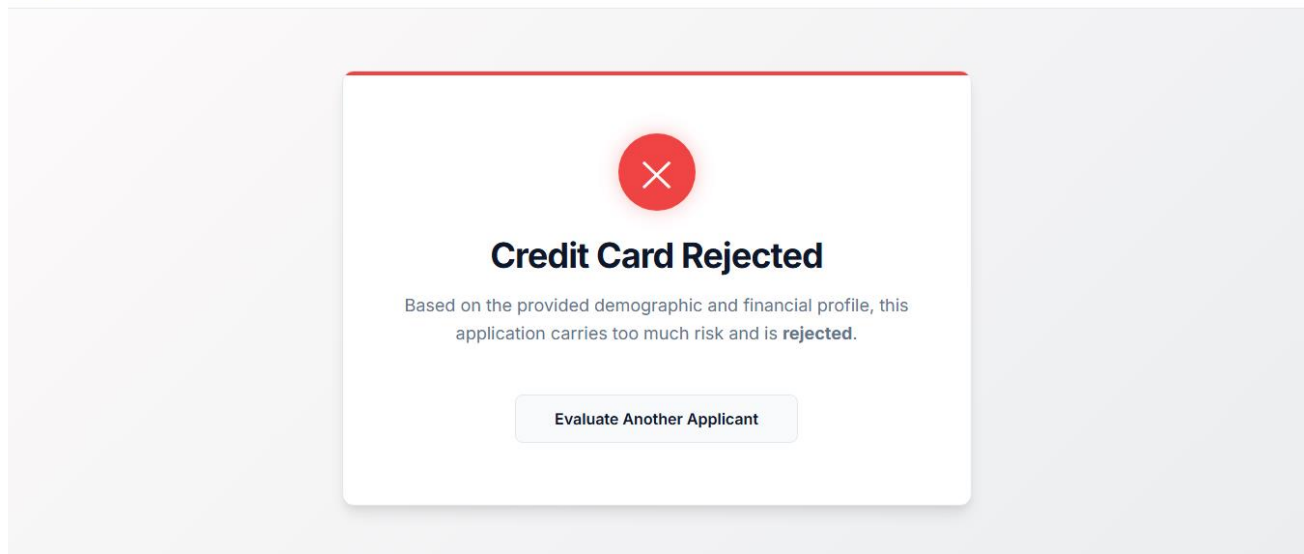
Once the user submits the form:

- The backend processes the data.
- The machine learning model predicts approval status.
- The result is displayed as:

Credit Card Approved

or

Credit Card Rejected

SkillWallet Internship

Additional prediction confidence or probability can also be displayed if implemented.

Milestone 5: IBM Watson Cloud Deployment

The final milestone focuses on deploying the trained machine learning model on IBM Watson Machine Learning. Cloud deployment allows users and financial institutions to access the prediction service remotely without requiring a local machine learning environment.

The deployment exposes the trained model as a REST API, enabling seamless integration with the Flask application.

Activity 5.1: Preparing the Model

Before deployment:

- Save the trained model.
- Save preprocessing objects.
- Verify dependencies.
- Prepare deployment package.

Activity 5.2: IBM Watson Machine Learning Deployment

Deployment involves:

- Creating an IBM Cloud account.
- Creating a Watson Machine Learning service.
- Uploading the trained model.
- Creating an online deployment.
- Generating API credentials.
- Testing prediction endpoints.

Activity 5.3: Integrating Flask with IBM Watson

The Flask application communicates with IBM Watson by:

- Sending applicant information through REST API requests.
- Receiving prediction responses.
- Displaying approval or rejection status to the user.

This enables scalable cloud-based predictions accessible from any device with internet connectivity.

Activity 5.4: Testing and Validation

The deployed application is tested using multiple applicant records to verify:

- Prediction accuracy
- Response time
- API connectivity
- User interface functionality
- End-to-end workflow

Successful testing confirms that the system can reliably automate the credit card approval process in real-world scenarios.

Milestone 6: Conclusion

The Credit Card Approval Prediction System successfully automates the credit card approval process using machine learning. By evaluating applicant financial and demographic information, the system predicts approval decisions accurately while reducing manual effort and processing time. The integration of Flask provides an easy-to-use web interface, while IBM Watson Machine Learning enables cloud deployment for scalable real-time predictions. Future enhancements include integrating explainable AI techniques, adding fraud detection modules, incorporating deep learning models, and retraining the system with continuously updated banking datasets to further improve prediction accuracy and reliability.

